

Паспорт проекта

Содержание

1. Состав решения.....	3
2. Архитектурная схема решения.....	4
3. Реализация обязательных требований.....	6
4. Ревью работы и потенциальное масштабирование	7

1. Состав решения

Операционная система: Ubuntu 24.04

Kubernetes-окружение: Kubernetes v1.31.0, развернут с помощью Kind v0.20.0 с пробросом портов 80/443 на хост

Gateway API: Envoy Gateway v1.0.0 (Gateway Controller, реализации GatewayClass, Gateway, HTTPRoute)

Веб-приложение: Nginx HTTP Server

Мониторинг: Prometheus Stack (Helm-чарт kube-prometheus-stack v65.x, включая Prometheus Operator и Grafana)

Логирование: Filebeat 8.11.0, развернут как DaemonSet с монтированием хостового пути /var/log/pods

Автоматизация: Bash-скрипт deploy.sh, включая скрипт проверки утилит scripts/install_deps.sh, Helm v3 и Kubectl

2. Архитектурная схема решения

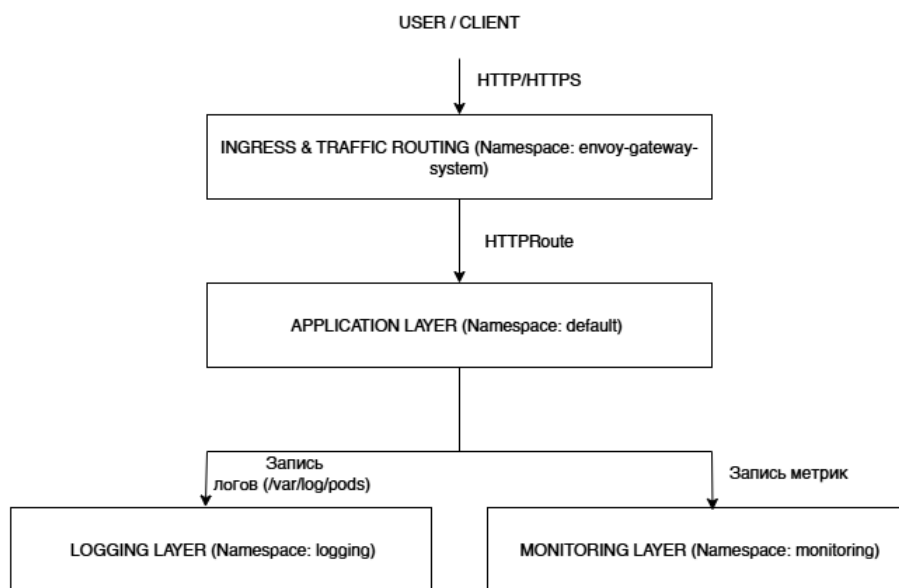


Рис. 1. Архитектурная схема решения

Состав решения:

1. Ingress & Traffic Routing (Envoy Gateway)

Принимает внешний входящий трафик, используется современный стандарт Kubernetes Gateway API (Gateway и HTTPRoute), который маршрутизирует запросы внутрь кластера на Service приложения.

2. Application Layer (Веб-приложение)

Контейнеры с Nginx обрабатывают HTTP-запросы от Envoy Proxy. При обработке запроса Nginx пишет текстовые access- и error-логи в поток stdout/stderr, которые со стороны Kubernetes сохраняются на хост-ноде в пути /var/log/pods/.

3. Logging Layer (Сбор логов)

Filebeat запущен как DaemonSet, а именно по одному экземпляру на каждую ноду кластера. Через hostPath монтируется в папку /var/log/pods, находит файлы логов именно пода web-app, декодирует CRI-формат containerd

и отправляет готовые структурированные события в вывод (output.console или Elasticsearch)

4. Monitoring & Observability (Метрики)

Prometheus Operator находит целевые поды через CRD ServiceMonitor. Prometheus опрашивает эндпоинты метрик приложений/Envoy Gateway и строит временные ряды, которые визуализируются в Grafana.

3. Реализация обязательных требований

В рамках Kubernetes-окружения и автоматизации развертывания в корневой папке был создан `deploy.sh`, проверяющий наличие зависимостей (`docker`, `kind`, `kubectrl`, `helm`), разворачивает локальный кластер Kind с конфигом проброса портов, последовательно применяет Manifest-файлы и Helm-чарты, ожидая полной готовности Pods. Использование Kind обеспечивает легковесное, повторно воспроизводимое и независимое от облачных провайдеров Kubernetes-окружение на базе Docker в Ubuntu 24.04

В качестве веб-приложения был развернут Deployment с публичным образом `nginx:alpine` в namespace `default`. Приложение генерирует стандартные access-логи HTTP-запросов в `stdout/stderr`

Была сделана публикация через Gateway API, установлен Envoy Gateway Controller, созданы CRD-ресурсы: `GatewayClass` (`eg`), `Gateway` (слушающий порт 80) и `HTTPRoute` (`web-app-route`), связывающий входящий HTTP-трафик со службой `web-app-service`.

Для мониторинга был установлен `kube-prometheus-stack` через Helm. Для веб-приложения настроен манифест `ServiceMonitor`, считывающий метрики приложения и контейнеров.

Было добавлено логирование, развернут DaemonSet `Filebeat` в namespace `logging`. Настроено монтирование директории узла `/var/log/pods` и обогащение логов метаданными Kubernetes (`Pod Name`, `Namespace`, `Container Name`) через `ServiceAccount` и `ClusterRole`. Чтение логов напрямую с файлов ноды гарантирует сбор логов без модификации кода приложения и без потери производительности.

Также дополнительно был внедрен скрипт `install_deps.sh`, который перед началом деплоя проверяет наличие всех необходимых утилит CLI и прав пользователя Docker, предотвращая падение установки на середине.

4. Ревью работы и потенциальное масштабирование

Главной сильной стороной проекта является абсолютная автономность и воспроизводимость на ОС Ubuntu 24.04 без внешних коммерческих зависимостей. Архитектура построена на актуальных стандартах CNCF — Kubernetes Gateway API и Prometheus Operator. Весь стек автоматизирован идемпотентным Bash-скриптом, который самостоятельно подготавливает локальный Kind-кластер и разворачивает полный Observability-стек.

Наиболее сложной задачей был выбор и корректная настройка Gateway API и согласование ролей RBAC для Filebeat в минималистичном локальном кластере. Приходилось выбирать между традиционным Nginx Ingress Controller и новым Envoy Gateway API, выбор был сделан в пользу более перспективного Gateway API. Также потребовалось тонко настроить ServiceMonitor и фильтрацию селекторов в Helm-чарте Prometheus, чтобы метрики приложения подхватывались автоматически.

Предложения по дальнейшему развитию:

- Перенаправление логов из Filebeat в централизованный кластер Loki или OpenSearch для долгосрочного хранения, индексации и поиска логов через Grafana;
- Перевод развертывания инфраструктурных манифестов и Helm-чартов с локальных скриптов на GitOps-оператор для декларативного управления состоянием кластера прямо из Git-репозитория;
- Добавление сквозного трассирования OpenTelemetry для анализа задержек (Latency) в цепочках микросервисов, что критически важно для высоконагруженных телеком-платформ.